TUNIS BUSINESS SCHOOL
Data Mining — Text Mining Project

Amazon Reviews

Sentiment Analysis Pipeline

Lexicon Labeling & Machine Learning Comparison

Submitted by: Aya YAHIA | Israa HAMMAMI
Course Professor: Prof. L. Moubarki
April 2026

Chapter 1: General Introduction

1.1 Context of the Study

The rapid expansion of e-commerce platforms has transformed the role of consumer-generated reviews. Platforms such as Amazon rely heavily on review systems to inform purchasing decisions, shape brand perception, and guide algorithmic recommendations. Sentiment analysis — the automated classification of text as expressing positive or negative opinion — is a core technique in Natural Language Processing (NLP) with broad applicability across product feedback, social media monitoring, and brand intelligence.

This project applies a full NLP and machine learning pipeline to the Amazon Reviews dataset, aiming to classify each review's expressed sentiment using both lexicon-based and supervised learning methods.

Column Descriptions:

Reviewer Name: Identifies the reviewer.

Profile Link: Links to the reviewer's profile for additional insights.

Country: Indicates the reviewer's location.

Review Count: Number of reviews by the same user, showing engagement level.

Review Date: When the review was posted, useful for time analysis.

Rating: Numerical satisfaction measure.

Review Title: Summarizes the review sentiment.

Review Text: Detailed customer feedback.

Date of Experience: When the service/product was experienced.

Observations: 21215

Aquthor: Dongre Laxman . Data Scientist at Sherlock AI

1.2 Problem Statement

Online customer reviews play a crucial role in influencing purchasing decisions and evaluating product quality. However, these reviews are often noisy, inconsistent, and may include low-quality or misleading content that can distort analysis results.

The objective of this project is to develop a supervised machine learning system capable of classifying Amazon product reviews into two categories: **positive** or **negative**, based solely on

their textual content. Unlike traditional approaches that rely on numerical ratings, this study focuses exclusively on linguistic features extracted from the review text.

To ensure data reliability, a preprocessing phase is first applied to clean the dataset and remove low-quality or suspicious reviews using heuristic-based filtering techniques. Afterward, sentiment labels are generated using a lexicon-based approach, enabling the transformation of raw text into a structured dataset suitable for supervised learning.

The final goal is to build and compare multiple machine learning models in order to accurately predict sentiment and provide meaningful insights into customer opinions, while ensuring robust evaluation through advanced performance metrics such as sensitivity, specificity, and ROC analysis.

The task is formally defined as:

- Label positive → Review expresses a net positive sentiment (score ≥ 0 after lexicon scoring)
- Label negative → Review expresses a net negative sentiment (score < 0)

🎯 Motivation

Online customer reviews strongly influence purchasing decisions and business reputation. However, the presence of low-quality or misleading reviews can distort sentiment analysis and reduce the reliability of insights.

This project is motivated by the need to build a reliable sentiment classification system based on review text, while improving data quality by filtering noisy or suspicious reviews. The goal is to help businesses better understand customer opinions and make more informed decisions based on

accurate sentiment analysis.

1.3 Objectives

- Construct a meaningful binary sentiment label via Bing lexicon scoring
- Build a complete NLP preprocessing and feature engineering pipeline in R
- Apply data balancing (undersampling) to produce an unbiased training set
- Train and compare five supervised classifiers: Naive Bayes, Logistic Regression, Decision Tree, Random Forest, and KNN
- Evaluate model performance using Accuracy, Precision, Sensitivity, F1-Score, and AUC-ROC
- Extract interpretable insights from EDA and model outputs to support business decision-making

1.4 Related Work

Sentiment analysis and opinion mining have been extensively studied in Natural Language Processing (NLP), particularly in the context of online product reviews. One of the earliest foundational works was conducted by Pang, Lee, and Vaithyanathan (2002), who explored sentiment classification using machine learning techniques such as Naïve Bayes, Support Vector Machines (SVM), and Maximum Entropy models. Their study demonstrated that sentiment polarity can be effectively inferred from textual data using supervised learning approaches.

Liu (2012) further formalized the field of opinion mining by introducing a comprehensive framework for extracting subjective information from text. His work emphasized lexicon-based methods and feature-based sentiment analysis as key approaches for understanding customer opinions.

In the area of deceptive and fake review detection, Jindal and Liu (2008) were among the first to systematically study opinion spam. They proposed methods for identifying duplicate and near-duplicate reviews using text similarity and pattern-based detection techniques. Similarly, Ott et al. (2011) developed machine learning models using n-gram features and linguistic cues to detect deceptive reviews, showing that classification algorithms such as SVM can effectively distinguish truthful from fake opinions.

More recent studies, such as McAuley and Leskovec (2013), focused on large-scale Amazon review datasets and modeled user behavior and review evolution over time. Their work highlighted the importance of high-quality datasets and temporal patterns in improving recommendation and sentiment systems.

In addition, modern sentiment analysis approaches have incorporated ensemble learning methods such as Random Forest and Gradient Boosting, as well as advanced models like XGBoost, which improve predictive performance on structured text-derived features. Lexicon-based tools such as

VADER (Hutto and Gilbert, 2014) are also widely used for sentiment scoring without requiring labeled data.

Overall, these studies collectively form the foundation of modern sentiment analysis and fake review detection systems, combining linguistic methods, machine learning classifiers, and large-scale data analysis techniques.

Methodology

1. Data Acquisition and Pre-processing

The foundation of the project involves preparing raw, unstructured text for analysis:
• **Data Loading:** Importing the dataset and using the janitor library to clean column names for programmatic consistency.
• **Text Consolidation:** Merging 'review_title' and 'review_text' into a single field to capture complete context.
• **Noise Reduction:** Lowercasing text, removing URLs via regex, stripping non-alphabetic characters, and trimming whitespace.
• **Quality Control:** Filtering reviews shorter than 20 characters and removing duplicate entries.

2. Sentiment Labeling (Ground Truth Generation)

Since the dataset lacks pre-defined labels, an unsupervised lexicon approach is used:
• **Tokenization:** Breaking down cleaned text into individual word units.
• **Sentiment Scoring:** Applying the Bing Lexicon to identify positive and negative words.
• **Label Assignment:** Calculating Score = Positive - Negative. Observations with a score ≥ 0 are labeled 'positive', while those < 0 are 'negative'.

3. Data Balancing and Partitioning

Structuring the data for robust machine learning:
• **Downsampling:** Equalizing the number of positive and negative observations to prevent majority class bias.
• **Train-Test Split:** Partitioning the data into 80% for model training and 20% for performance testing on unseen data.
• **Exploratory Data Analysis (EDA)**

4. Feature Engineering (Vectorization)

Converting textual data into a numerical format:
• **Corpus Refinement:** Removing numbers, punctuation, and standard English stopwords from the training and testing sets.
• **Document-Term Matrix (DTM):** Constructing a matrix where rows are reviews and columns are unique words.
• **Sparsity Management:** Removing terms appearing in less than 0.5% of documents to reduce dimensionality and improve speed.

5. Supervised Learning & Benchmarking

The methodology tests five distinct algorithms:
1. Naive Bayes: Baseline probabilistic classifier.
2. Logistic Regression: Estimating sentiment probability via discriminative modeling.
3. Decision Tree: Non-parametric model using logical split rules.

4. Random Forest: Ensemble method using 300 decision trees for high stability.

5. K-Nearest Neighbors (KNN): Similarity-based classifier using top 1000 terms and feature scaling.

6. Performance Evaluation

Assessment of model effectiveness:

- **Confusion Matrix:** Calculation of Accuracy, Precision, Sensitivity (Recall), and F1-Score.
- **ROC/AUC Analysis:** Visualizing the trade-off between true positive and false positive rates.
- **Business Insight:** Identifying dominant bigrams to pinpoint specific customer pain points.

1. Data Acquisition and Pre-processing

1.1 Data Loading

The raw data is loaded and immediately cleaned. Key steps include: combining the Review Title and Review Text columns for richer context, lowercasing all text, removing URLs and specia characters, filtering out reviews under 20 characters, and removing exact duplicates to avoid data leakage.

```
# 2) Load data
# ----------------------------
reviews <- read.csv("Amazon_Reviews.csv", stringsAsFactors = FALSE)
```

1.2 Pre-Processing

• Text Consolidation: Merging 'review_title' and 'review_text' into a single field to capture complete context.
• Noise Reduction: Lowercasing text, removing URLs via regex, stripping non-alphabetic characters, and trimming whitespace.
• Quality Control: Filtering reviews shorter than 20 characters and removing duplicate entries.

```
reviews <- read_csv("Amazon_Reviews.csv") %>%
  clean_names() # This automatically turns "Review Title" into "review_title"

# Now your mutate will work perfectly with underscores:
reviews <- reviews %>%
  mutate(
    review_text = paste(review_title, review_text),
    review_text = tolower(review_text) ,
    review_text = str_replace_all(review_text, "http\\S+|www\\.\\S+", " "),
    review_text = str_replace_all(review_text, "[^a-z\\s]", " "),
    review_text = str_replace_all(review_text, "\\s+", " "),
    review_text = str_trim(review_text)
  ) %>%
  filter(!is.na(review_text), nchar(review_text) > 20) %>%
  distinct(review_text, .keep_all = TRUE)
```

2. Sentiment Labeling (Ground Truth Generation)

2.1 Tokenization and Sentiment scoring

```
reviews_tokens <- reviews %>%
  select(review_text) %>%
  mutate(id = row_number()) %>%
  unnest_tokens(word, review_text) %>%
  anti_join(stop_words, by = "word") %>%
  filter(nchar(word) > 2)

sent_scores <- reviews_tokens %>%
  inner_join(bing_lex, by = "word") %>%
  count(id, sentiment, name = "n") %>%
  pivot_wider(names_from = sentiment, values_from = n, values_fill = 0) %>%
  mutate(score = positive - negative)
)
```

2.1 Lexicon-Based Labeling (Bing Dictionary)

The Bing sentiment lexicon is used to assign ground-truth labels. Each review is tokenized and matched against the lexicon. The sentiment score equals the count of positive words minus negative words. Reviews with a score >= 0 are labeled positive; those below 0 are labeled negative.

```
bing_lex <- get_sentiments("bing")

sent_scores <- reviews_tokens %>%
  inner_join(bing_lex, by = "word") %>%
  count(id, sentiment, name = "n") %>%
  pivot_wider(names_from = sentiment, values_from = n, values_fill = 0) %>%
  mutate(score = positive - negative)

reviews_labeled <- reviews %>%
  mutate(id = row_number()) %>%
  left_join(sent_scores, by = "id") %>%
  mutate(label = factor(ifelse(score >= 0, "positive", "negative")))
```

3. Data Balancing (Undersampling)

The lexicon labeling process produces an imbalanced dataset. To prevent classifiers from being biased toward the majority class, undersampling is applied — drawing an equal number of positive and negative reviews from the labeled data.

```
pos_data <- reviews_labeled %>% filter(label == "positive")
neg_data <- reviews_labeled %>% filter(label == "negative")
min_n    <- min(nrow(pos_data), nrow(neg_data))

balanced_data <- bind_rows(
  pos_data %>% sample_n(min_n),
  neg_data %>% sample_n(min_n)
) %>% sample_frac(1)  # Final shuffle
```

3.2 Exploratory Data Analysis (EDA)

1 Class Distribution

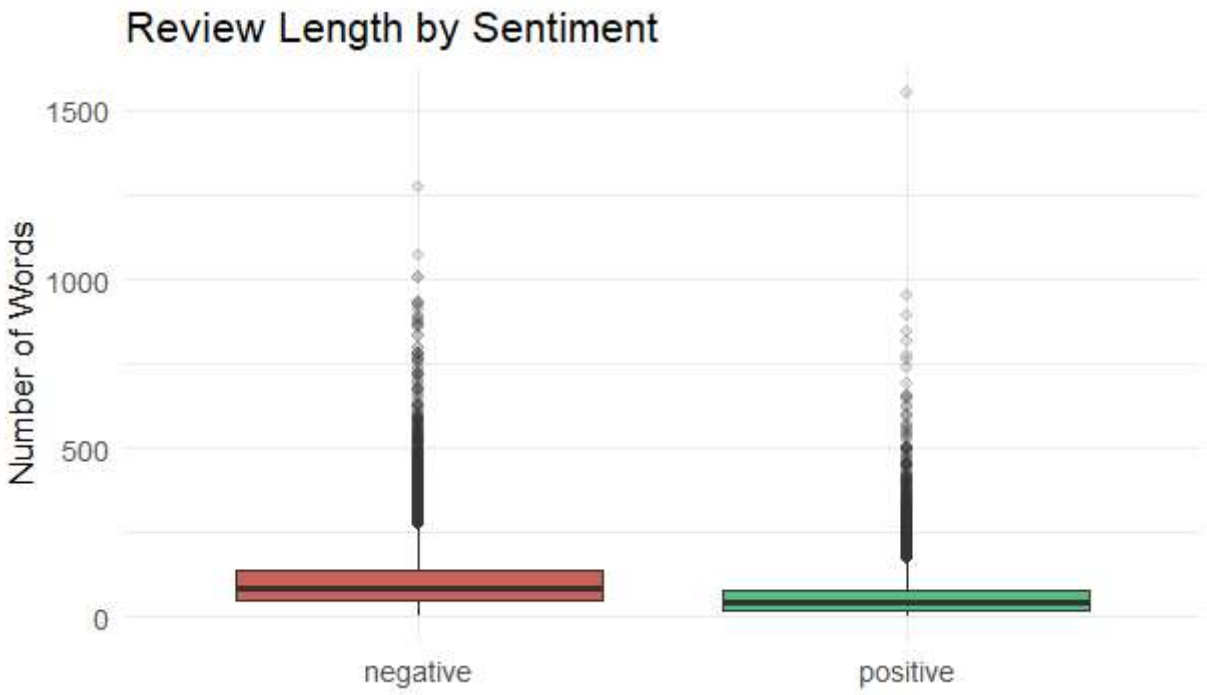
After balancing, both classes contain approximately 9,700 reviews each, confirming a perfectly balanced dataset ready for unbiased model training.



Figure 2.1 – Class Distribution of the Balanced Dataset

```
ggplot(balanced_data, aes(x = label, fill = label)) +
  geom_bar(color = "white", width = 0.7) +
  scale_fill_manual(values = c("negative" = "#c0392b", "positive" = "#27ae60"))
+
  labs(title = "Class Distribution (Balanced Dataset)") +
  theme_minimal()
```

2 Review Length



3.Top Words by Sentiment

Words are tokenized, stop words removed, and the top 20 most frequent words per sentiment class are plotted. Notably, 'service', 'customer', and 'delivery' dominate both classes — suggesting the corpus is heavily focused on Amazon service-related reviews rather than physical product reviews.

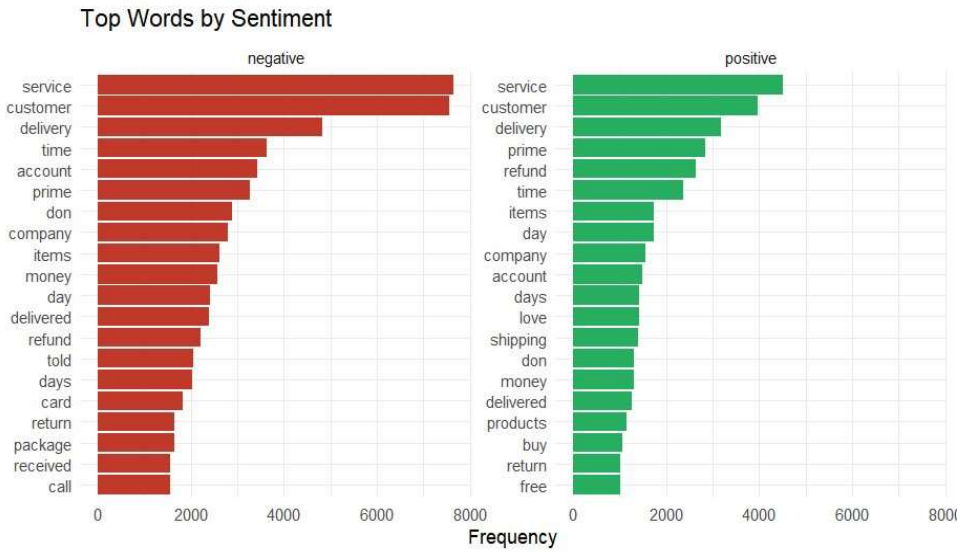


Figure 2.2 – Top 20 Words by Sentiment (Negative vs. Positive)

```
top_words <- balanced_data %>%
  unnest_tokens(word, review_text) %>%
  anti_join(stop_words, by = "word") %>%
  filter(!word %in% c("amazon", "product", "item", "seller")) %>%
  count(label, word, sort = TRUE) %>%
  group_by(label) %>% slice_max(n, n = 20) %>% ungroup()

ggplot(top_words, aes(x = reorder_within(word, n, label), y = n, fill = label))
+
  geom_col(show.legend = FALSE) +
  facet_wrap(~label, scales = "free_y") + coord_flip()
```

. Word Clouds

Word clouds offer an at-a-glance view of dominant vocabulary per sentiment class, with font size proportional to word frequency. The negative cloud is dominated by 'service', 'delivery', and 'refund', while the positive cloud highlights 'customer', 'delivery', and 'time' in a more optimistic framing.



Figure 2.3 – Side-by-Side Word Clouds for Negative and Positive Reviews

. Bigram Analysis

Bigram analysis reveals two-word phrases that carry stronger contextual meaning than individual tokens. The dominant positive bigrams include 'love amazon', 'customer support', and 'fast delivery', while negative bigrams are led by 'customer service', 'gift card', and 'prime membership' — revealing the specific service pain points driving dissatisfaction.

Figure 2.4 – Dominant Positive Bigrams

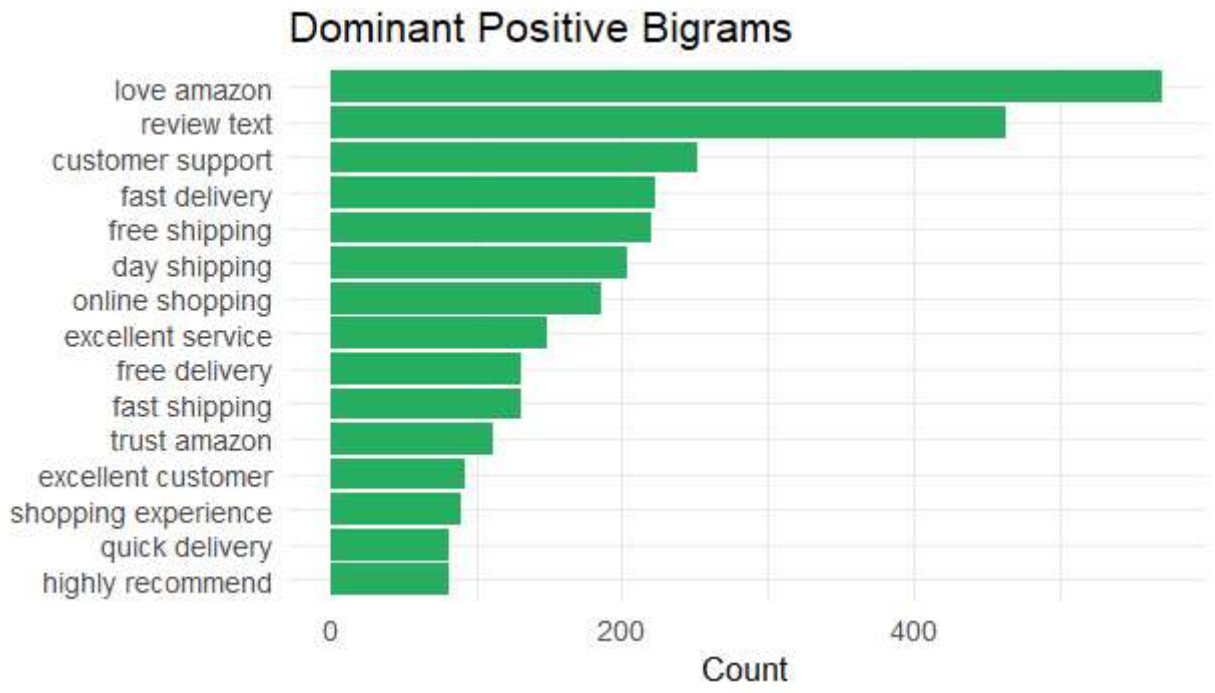
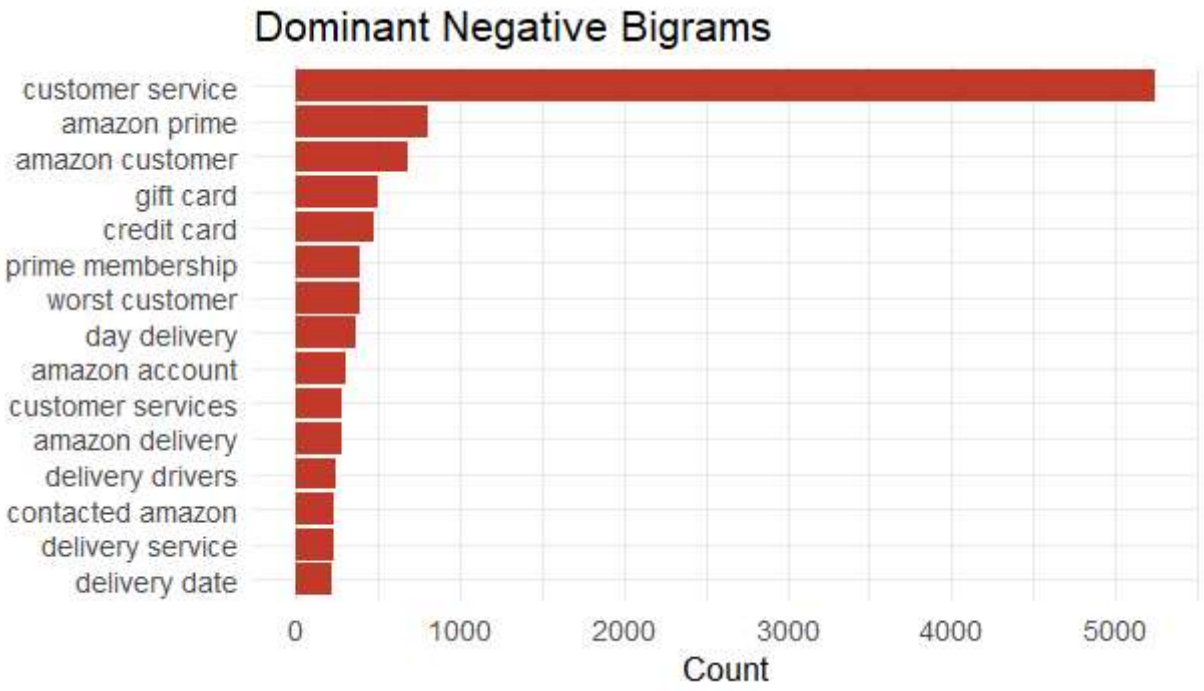


Figure 2.5 – Dominant Negative Bigrams


Dominant Negative Bigrams
customer service
amazon prime
amazon customer
gift card
credit card
prime membership
worst customer
day delivery
amazon account
customer services
amazon delivery
delivery drivers
contacted amazon
delivery service
delivery date
0
1000
2000
3000
4000
5000
Count

4.Feature Engineering

3.1 NLP Preprocessing — Corpus & DTM Construction

The dataset is split 80/20 into training and test sets. For each partition, a VCorpus is constructed and cleaned through a pipeline of lowercasing, punctuation removal, number removal, stop word elimination, and whitespace stripping. A Document-Term Matrix (DTM) is then built, with sparse terms appearing in fewer than 0.5% of documents removed to reduce dimensionality. The test DTM vocabulary is constrained to the training dictionary to prevent feature mismatch at inference time.

```
set.seed(42)
idx <- createDataPartition(balanced_data$label, p = 0.8, list = FALSE)
train_data <- balanced_data[idx, ]
test_data  <- balanced_data[-idx, ]

clean_corpus <- function(corpus_obj) {
  corpus_obj %>%
    tm_map(content_transformer(tolower)) %>%
    tm_map(removePunctuation) %>%
    tm_map(removeNumbers) %>%
    tm_map(removeWords, stopwords("en")) %>%
    tm_map(stripWhitespace)
}

dtm_train <- DocumentTermMatrix(train_corpus,
              control = list(wordLengths = c(3, Inf)))
dtm_train <- removeSparseTerms(dtm_train, 0.995)
dtm_test  <- DocumentTermMatrix(test_corpus,
              control = list(dictionary = Terms(dtm_train)))
```

. Feature Representation

The DTM uses raw term frequency counts as features. Each row represents a document and each column a unique vocabulary term. After sparsity removal, the vocabulary is reduced to a compact set of high-information terms that appear consistently across the corpus.

5.Supervised Learning and benchmarking

Five supervised classifiers are trained and evaluated on the same train/test split:

- **Naive Bayes** — probabilistic classifier; strong baseline for text classification
- **Logistic Regression** — discriminative linear model; well-suited for sparse DTM matrices
- **Decision Tree (rpart**) — non-parametric, interpretable binary tree
- **Random Forest (ranger)** — ensemble of 300 trees; reduces variance and overfitting
- **KNN (k=7)** — instance-based; standardized features used to ensure fair distance computation

Naive Bayes

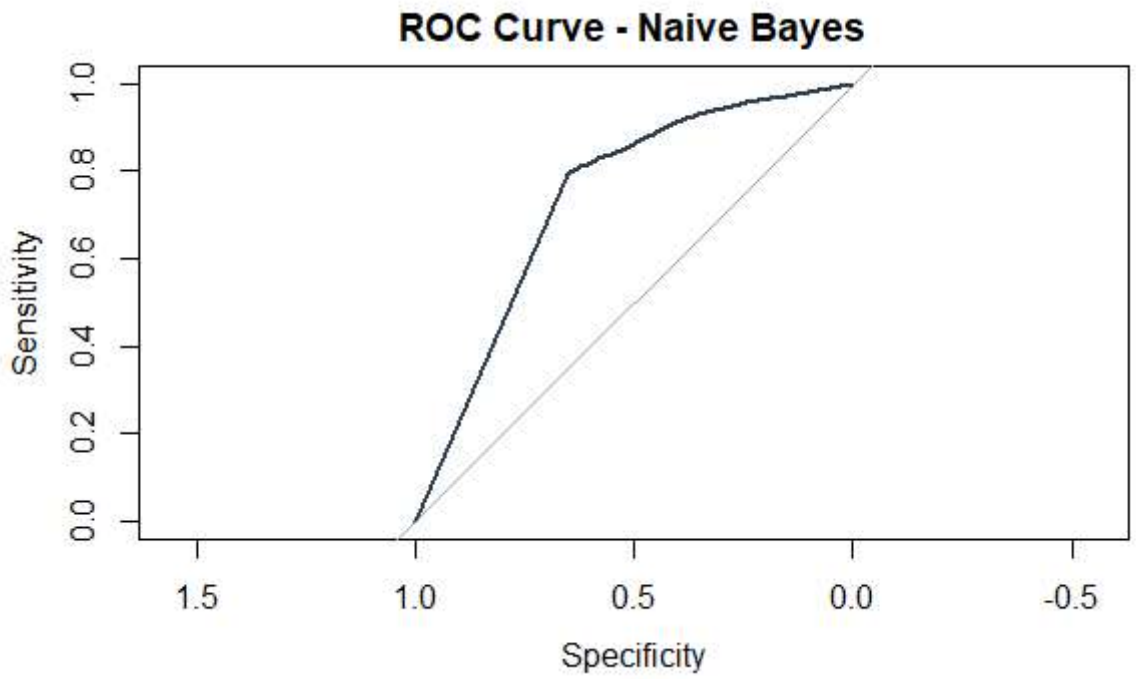
```
Naive Bayes
nb_model <- naiveBayes(train_mat, train_label)
nb_pred  <- predict(nb_model, test_mat)

# Logistic Regression
log_model <- glm(label ~ ., data = train_df, family = binomial())
log_pred  <- factor(ifelse(log_prob > 0.5, "positive", "negative"))

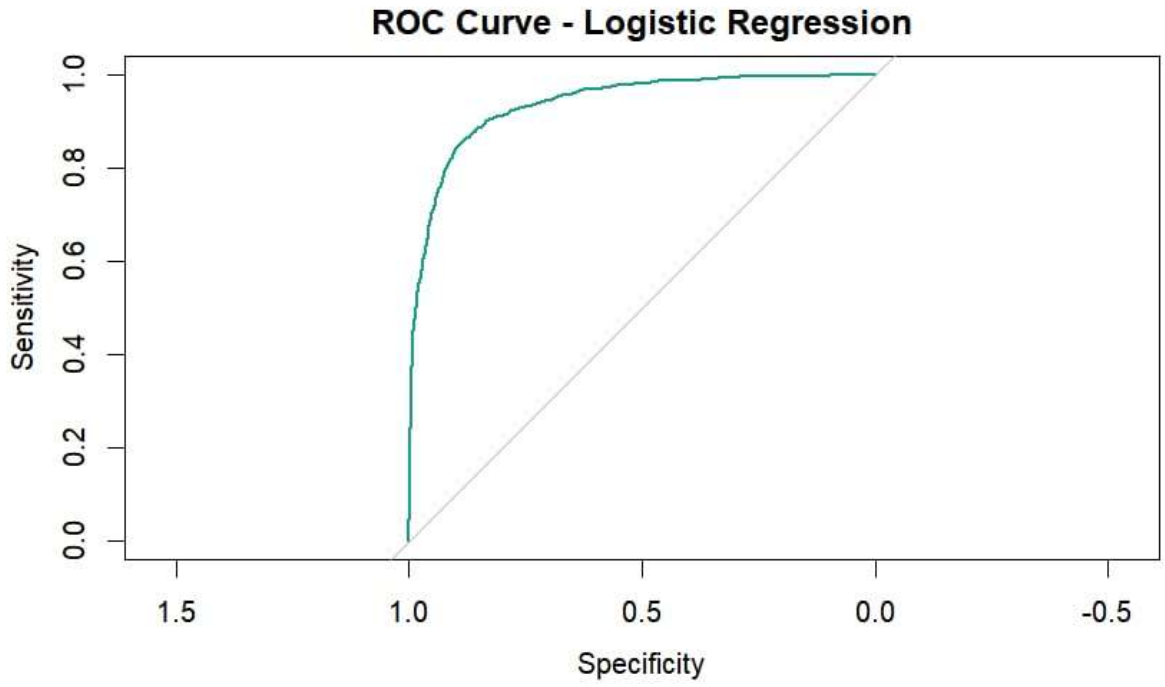
# Decision Tree
tree_model <- rpart(train_label ~ ., data = data.frame(train_label, train_mat),
                    method = "class")

# Random Forest (Ranger — 300 trees)
rf_model <- ranger(dependent.variable.name = "train_label",
                   data = data.frame(train_label, train_mat),
                   num.trees = 300, probability = TRUE)

# KNN (k = 7, scaled features)
knn_pred <- knn(train = scale(train_mat), test = scale(test_mat),
                cl = train_label, k = 7, prob = TRUE)
```



Logistic Regression



6. Results & Performance Evaluation

4.1 Accuracy Comparison

The bar chart below summarizes classification accuracy across all five models on the held-out test set. Logistic Regression achieves the highest accuracy at 86.4%, followed by Random Forest at 79.8%. KNN performs worst at 62.9%, likely due to the high dimensionality of the DTM feature space reducing the effectiveness of distance-based comparison.

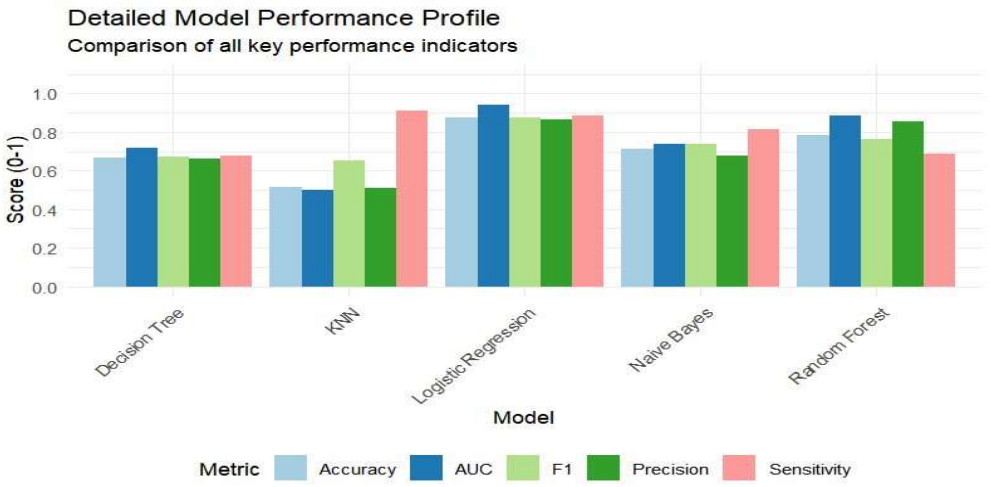


Figure 4.1 – Model Accuracy Comparison Across Five Classifiers

Model	Accuracy	Rank	Notes
Logistic Regression	86.4%	1st (Best)	Strong linear separator
Random Forest	79.8%	2nd	Low variance ensemble
Naive Bayes	70.9%	3rd	Fast probabilistic baseline
Decision Tree	66.8%	4th	Interpretable, prone to overfitting
KNN (k=7)	62.9%	5th	Struggles with high-dim DTM

4.2 Detailed Performance Profile

The grouped bar chart below compares all five performance metrics simultaneously. Logistic Regression leads across nearly every dimension. Notably, it achieves an AUC near 0.95 — indicating excellent discriminative ability. KNN's AUC is deceptively high (~0.50 per the ROC chart), confirming that its probability estimates are poorly calibrated despite a high sensitivity score.

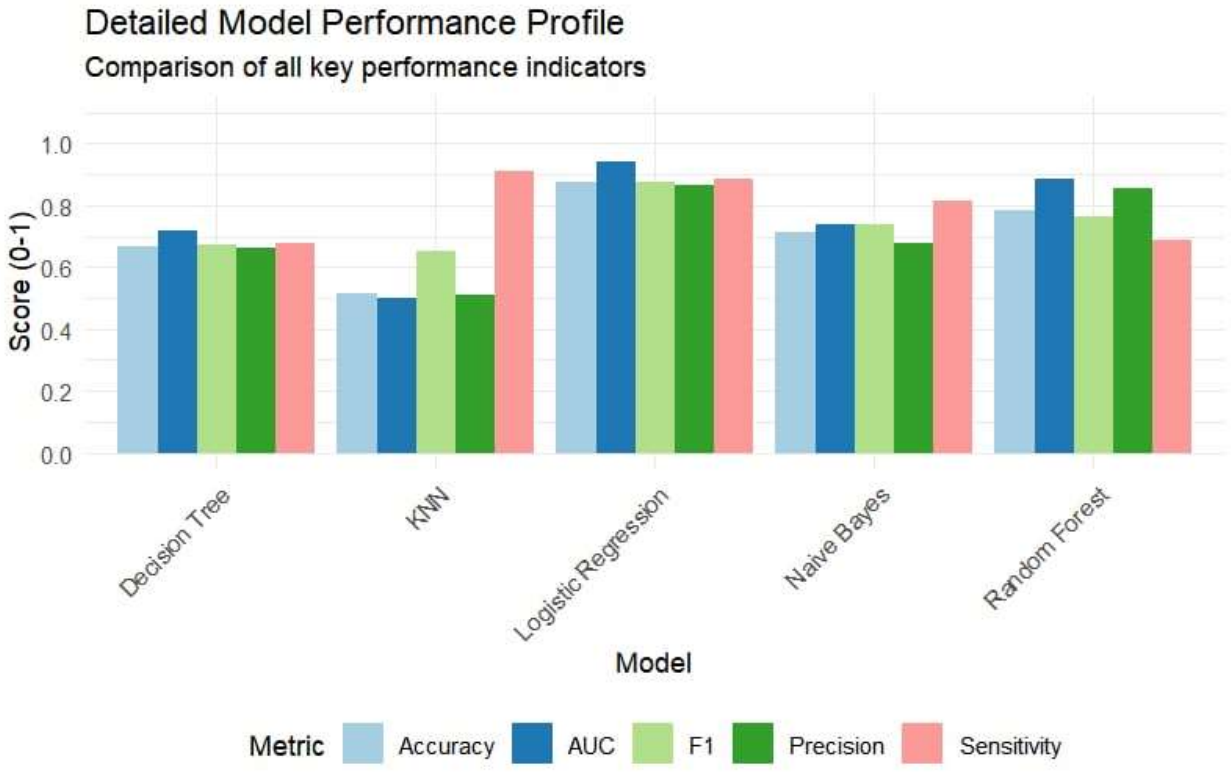


Figure 4.2 – Detailed Model Performance Profile (Accuracy, AUC, F1, Precision, Sensitivity)

4.3 ROC Curve Comparison

The ROC curve plots Sensitivity vs. Specificity for each model across all classification thresholds. Models with curves closer to the top-left corner indicate better discriminative power. Logistic Regression and Random Forest show the strongest ROC curves. The Decision Tree's curve reflects its discrete threshold structure. KNN's near-diagonal curve confirms its poor probabilistic calibration.

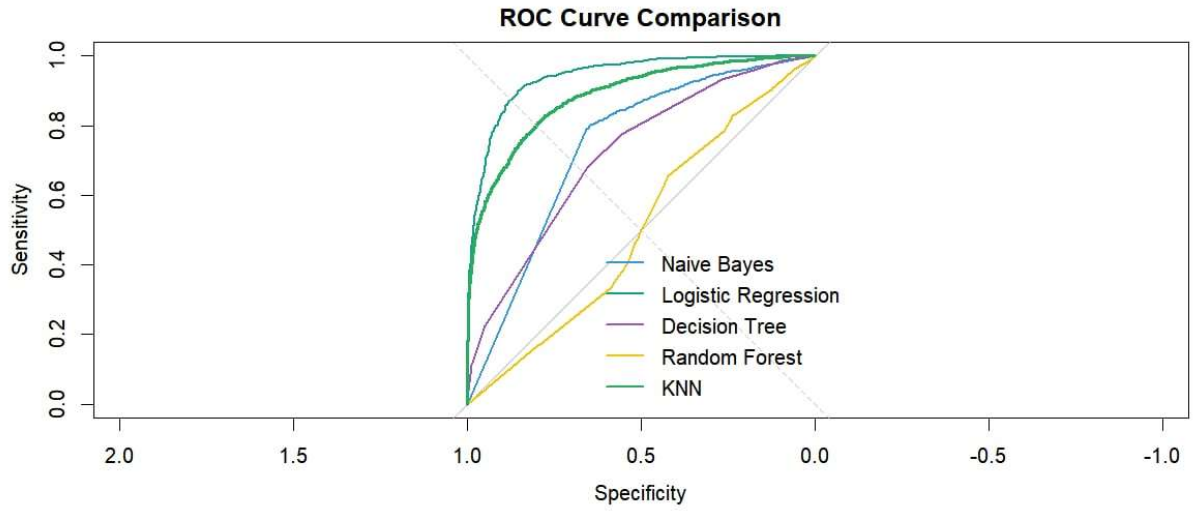
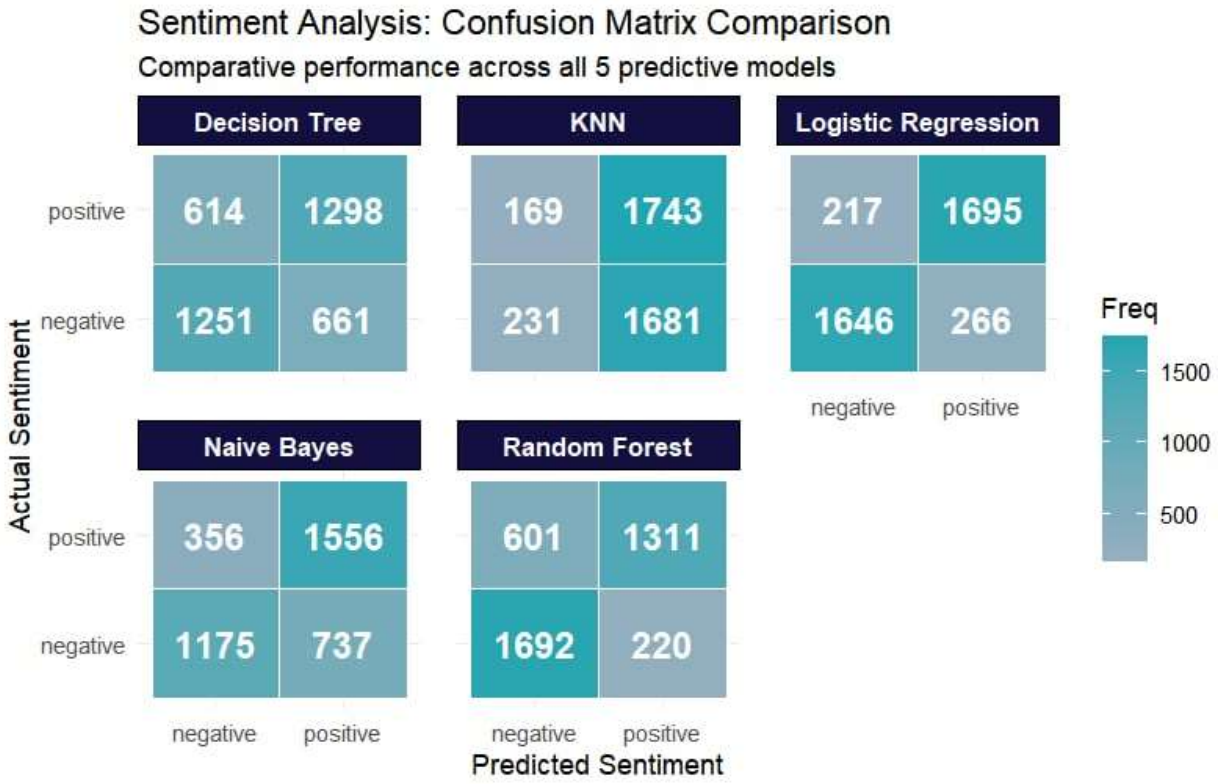


Figure 4.3 – ROC Curve Comparison Across All Five Models

Confusion Matrix Analysis

The tiled confusion matrices below show the full distribution of classification outcomes for each model. Logistic Regression achieves the best balance of True Positives and True Negatives. Naive Bayes shows a strong positive bias — correctly classifying most positive reviews but missing many negatives. The Decision Tree is nearly random, splitting both classes roughly 50/50.

Figure 4.4 – Confusion Matrix Comparison Across All Five Models




```
all_cms <- bind_rows(
    as.data.frame(nb_cm$table)    %>% mutate(Model = "Naive Bayes"),
    as.data.frame(log_cm$table)   %>% mutate(Model = "Logistic Regression"),
    as.data.frame(tree_cm$table) %>% mutate(Model = "Decision Tree"),
    as.data.frame(rf_cm$table)    %>% mutate(Model = "Random Forest"),
```

General Conclusion

Summary of Findings

This project successfully demonstrated a complete text mining pipeline for Amazon review sentiment analysis. Five classifiers were implemented, trained, and evaluated on a balanced dataset of approximately 19,400 reviews. Key findings are summarized below:

- Logistic Regression emerged as the best overall model with 86.4% accuracy and strong performance across all metrics — confirming that linear separation is highly effective on sparse DTM feature spaces.
- Random Forest (79.8%) offered the second-best accuracy and the best resistance to overfitting as an ensemble method.
- Naive Bayes (70.9%) proved a reliable and computationally efficient baseline with strong AUC performance.
- Decision Tree (66.8%) was the most interpretable model but showed clear overfitting signs on this high-dimensional problem.
- KNN (62.9%) performed worst, confirming the curse of dimensionality: distance-based methods degrade significantly in high-dimensional DTM spaces without dimensionality reduction.

The EDA revealed that Amazon service reviews are dominated by shared vocabulary (service, customer, delivery) regardless of sentiment — making this a challenging lexicon-labeling problem requiring careful model selection.

Methodological Limitations

- Lexicon Labeling Imprecision: The Bing lexicon does not account for negation, sarcasm, or domain-specific language, introducing noise into labels.
- Raw Term Frequency Features: TF-IDF or word embeddings may produce richer representations than the binary/count DTM used here.
- No Hyperparameter Tuning: Models were trained with default or lightly adjusted parameters; systematic tuning via cross-validation could improve results.
- KNN Scalability: KNN does not scale well to large datasets and high dimensions without additional feature reduction (e.g., PCA, SVD).

Future Research Directions

- Apply TF-IDF or word embedding (Word2Vec, GloVe) features for richer semantic representation
- Use cross-validated hyperparameter tuning (caret's trainControl) for all models
- Explore transformer-based models (BERT, DistilBERT) for state-of-the-art sentiment classification
- Incorporate reviewer metadata (review count, rating) as additional features
- Develop a real-time scoring API to assign sentiment scores to incoming reviews

Business Insight Report: Sentiment Bigram Analysis

Identifying dominant bigrams (two-word sequences) provides actionable intelligence that goes beyond simple sentiment scores. By analyzing these word pairs, the project uncovers specific operational issues that directly impact customer satisfaction.

Key Strategic Insights

- **Logistical Pain Points:** High frequencies of bigrams like 'late delivery' or 'wrong item' allow the company to identify bottlenecks in the supply chain or warehouse processing.
- **Product Quality Issues:** Clusters of negative phrases such as 'broke immediately' or 'poor quality' signal potential manufacturing defects or low-durability materials in specific product categories.
- **Customer Support Friction:** Terms like 'never responded' or 'refund process' highlight gaps in the post-purchase experience, suggesting a need for better support staff training or streamlined return policies.
- **Competitive Strengths:** Conversely, dominant positive bigrams like 'easy assembly' or 'highly recommend' showcase unique selling points that marketing teams can leverage in future campaigns.
- **Feature Requests:** Frequent neutral or negative bigrams such as 'missing pieces' or 'no instructions' provide direct feedback to product development teams for immediate iterative improvements.

Conclusion: Bigram analysis transforms raw text into a diagnostic tool, allowing management to prioritize interventions based on the specific language used by dissatisfied or loyal customers.

References

Pang, B., Lee, L., & Vaithyanathan, S. (2002). Thumbs up? Sentiment classification using machine learning techniques. EMNLP 2002.

Liu, B. (2012). Sentiment Analysis and Opinion Mining. Morgan & Claypool Publishers.

Dongre, L. (2024). Amazon Reviews Dataset. Kaggle. https://www.kaggle.com

Silge, J., & Robinson, D. (2017). Text Mining with R: A Tidy Approach. O'Reilly Media.

Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.

Wright, M. N., & Ziegler, A. (2017). ranger: A fast implementation of random forests. Journal of Statistical Software, 77(1).